

Relatório de Perfilamento e Análise

Otimização de Telemetria com Buffer Circular

Grupo 15

Junho de 2026

Resumo

Este relatório analisa a eficiência de duas estratégias para armazenamento e transmissão de amostras de sensores em sistemas embarcados: uma vertente ineficiente baseada em realocação dinâmica e deslocamento de memória, e uma vertente eficiente baseada em buffer circular de tamanho fixo. A comparação considera complexidade assintótica, estabilidade temporal, impacto no *heap* e comportamento sob latência de rede MQTT. Os gráficos foram gerados em um notebook Python reproduzível a partir de um benchmark controlado, complementando a instrumentação prevista no firmware ESP32.

1 Objetivo

O objetivo do projeto é demonstrar, de forma teórica e experimental, por que a gerência de telemetria em sistemas embarcados deve evitar estruturas que exigem realocação dinâmica ou deslocamento integral de memória a cada nova amostra. Em dispositivos como ESP32, a captura de sinais pode ocorrer em escala de microssegundos, enquanto a transmissão MQTT depende da rede e pode bloquear por milissegundos. Essa diferença temporal torna essencial separar a produção das amostras do consumo/transmissão.

O requisito central é comparar duas vertentes:

- **Vertente 1, ineficiente:** uso de coleção dinâmica com `realloc()` e `memmove()`, além de publicação MQTT síncrona por amostra.
- **Vertente 2, eficiente:** uso de buffer circular com índices `head` e `tail`, operações em tempo constante e envio em lote ou modelo produtor-consumidor.

2 Contexto do Sistema

O projeto analisado combina captura embarcada, mensageria MQTT, processamento em Python, persistência e visualização web. Após a revisão do código, o firmware eficiente em `hardware/src/main.cpp` foi atualizado para conter uma implementação explícita de `RingBuffer` com índices `head` e `tail`, atendendo diretamente ao requisito de buffer circular. O fluxo principal é:

ESP32 → MQTT/Mosquitto → Node-RED → Notebook Python → PostgreSQL →
FastAPI → React

No firmware eficiente, o ESP32 opera em modo promíscuo e captura quadros WiFi de gerenciamento. Cada ocorrência contém endereço MAC, RSSI, canal, frequência, tipo de quadro e contagem de aparições. Esses dados são agregados e publicados no tópico `esp32/wifi/scan`. A camada Node-RED valida e normaliza as mensagens antes do processamento em Python.

Também existe uma vertente complementar com sensor óptico no ESP8266, usada para eventos de entrada e saída. Ela reforça a mesma ideia arquitetural: sensores produzem eventos rapidamente, enquanto rede, banco e visualização devem consumir esses eventos de maneira desacoplada.

3 Metodologia

A comparação foi construída a partir de três fontes:

- leitura do requisito do projeto, que exige comparação entre deslocamento dinâmico e buffer circular;
- inspeção do código-fonte do projeto, especialmente `hardware_ineficiente/src/main.cpp`, `hardware/src/main.cpp` e os notebooks Python;
- execução de um notebook de perfilamento local que reproduz o comportamento assintótico das duas estratégias.

Como não havia, no pacote entregue, um arquivo CSV com medições seriais reais do ESP32, os gráficos deste relatório usam um **benchmark controlado em Python**. Ele não substitui a medição no hardware, mas evidencia a tendência algorítmica exigida: deslocar uma lista inteira cresce com N , enquanto atualizar o índice de um buffer circular se mantém constante.

Tabela 1: Comparação conceitual entre as vertentes.

Critério	Vertente ineficiente	Vertente eficiente
Inserção	<code>realloc()</code> + <code>memmove()</code>	escrita em <code>head</code>
Custo por amostra	$O(n)$	$O(1)$
Custo acumulado para N amostras	$O(N^2)$	$O(N)$
Memória	crescimento e fragmentação	bloco fixo previsível
Rede	publicação síncrona por amostra	lote ou produtor-consumidor
Risco operacional	<i>jitter</i> , perda e falha de alocação	latência estável e perda controlada

4 Vertente Ineficiente

A implementação ineficiente armazena cada nova amostra em uma lista dinâmica. A cada inserção, o vetor é realocado para o novo tamanho e todo o histórico é deslocado uma posição para manter a amostra mais recente na posição zero. O trecho abaixo resume o antipadrão presente em `hardware_ineficiente/src/main.cpp`.

```

1 InefficientSample *resizedSamples =
2   static_cast<InefficientSample *>(
3     realloc(inefficientSamples,
4             newCount * sizeof(InefficientSample)));

```

```

5
6 inefficientSamples = resizedSamples;
7
8 if (inefficientSampleCount > 0) {
9     memmove(inefficientSamples + 1,
10             inefficientSamples,
11             inefficientSampleCount * sizeof(InefficientSample));
12 }

```

Essa abordagem tem dois problemas. Primeiro, a operação `memmove()` copia uma quantidade de bytes proporcional ao número de amostras já armazenadas. Segundo, o uso repetido de `realloc()` pressiona o *heap*, aumentando a probabilidade de fragmentação e falhas de alocação. Em sistemas embarcados, isso aparece como aumento de latência, instabilidade temporal e eventual descarte de dados.

Além disso, a vertente ineficiente publica cada amostra individualmente via MQTT. Quando a rede está lenta, a rotina de publicação compete diretamente com a coleta, causando perda de amostras e *jitter*.

5 Vertente Eficiente com Buffer Circular

Na vertente eficiente, a estrutura de armazenamento tem tamanho fixo e é percorrida por índices. A inserção escreve na posição apontada por `head`; depois, `head` avança usando aritmética modular. Quando o buffer enche, a próxima inserção avança `tail` e sobrescreve o evento mais antigo, preservando as amostras mais recentes.

O firmware eficiente foi alterado para usar esse buffer circular entre a rotina de captura promíscua do WiFi e a rotina que drena os eventos para agregação. Assim, a interrupção apenas empilha um `ProbeEvent` em tempo constante, enquanto o processamento posterior consolida os dispositivos observados.

```

1 template <typename T, size_t Capacity>
2 class RingBuffer {
3 public:
4     bool push(const T &item) {
5         if (count == Capacity) {
6             tail = (tail + 1) % Capacity;
7             count--;
8             droppedCount++;
9         }
10
11         buffer[head] = item;
12         head = (head + 1) % Capacity;
13         count++;
14         return true;
15     }
16
17     bool pop(T &item) {
18         if (count == 0) {
19             return false;
20         }
21

```

```

22         item = buffer[tail];
23         tail = (tail + 1) % Capacity;
24         count--;
25         return true;
26     }
27
28 private:
29     T buffer[Capacity];
30     size_t head = 0;
31     size_t tail = 0;
32     size_t count = 0;
33     size_t droppedCount = 0;
34 };
35
36 static RingBuffer<ProbeEvent, PROBE_RING_BUFFER_CAPACITY>
    probeRingBuffer;

1 void wifiPromiscuousRx(void *buf, wifi_promiscuous_pkt_type_t
    type) {
2     // ... validacao do pacote e preenchimento do ProbeEvent ...
3     portENTER_CRITICAL_ISR(&probeRingMux);
4     probeRingBuffer.push(event);
5     portEXIT_CRITICAL_ISR(&probeRingMux);
6 }
7
8 void drainProbeQueue() {
9     ProbeEvent event;
10    for (;;) {
11        portENTER_CRITICAL(&probeRingMux);
12        const bool hasEvent = probeRingBuffer.pop(event);
13        portEXIT_CRITICAL(&probeRingMux);
14
15        if (!hasEvent) {
16            break;
17        }
18
19        aggregateProbeEvent(event);
20    }
21 }

```

Além do buffer circular de eventos, o projeto otimizado evita alocação dinâmica durante a janela de captura, usa memória estática para os dispositivos observados e publica os resultados em lote. Essa combinação remove o `realloc()`, elimina o deslocamento em massa e mantém o caminho crítico da captura previsível.

6 Análise Assintótica

Na vertente ineficiente, a i -ésima inserção precisa deslocar $i - 1$ elementos. Para inserir N amostras, o custo total é:

$$T(N) = \sum_{i=1}^N i = \frac{N(N+1)}{2}$$

Logo, o custo acumulado é $O(N^2)$ e o custo médio por inserção cresce linearmente com o tamanho do histórico.

No buffer circular, cada inserção executa uma escrita, uma soma, uma operação de módulo e no máximo uma atualização de contador. Essas operações não dependem de N :

$$T(N) = \sum_{i=1}^N 1 = N$$

Portanto, o custo acumulado é $O(N)$ e o custo por inserção é $O(1)$.

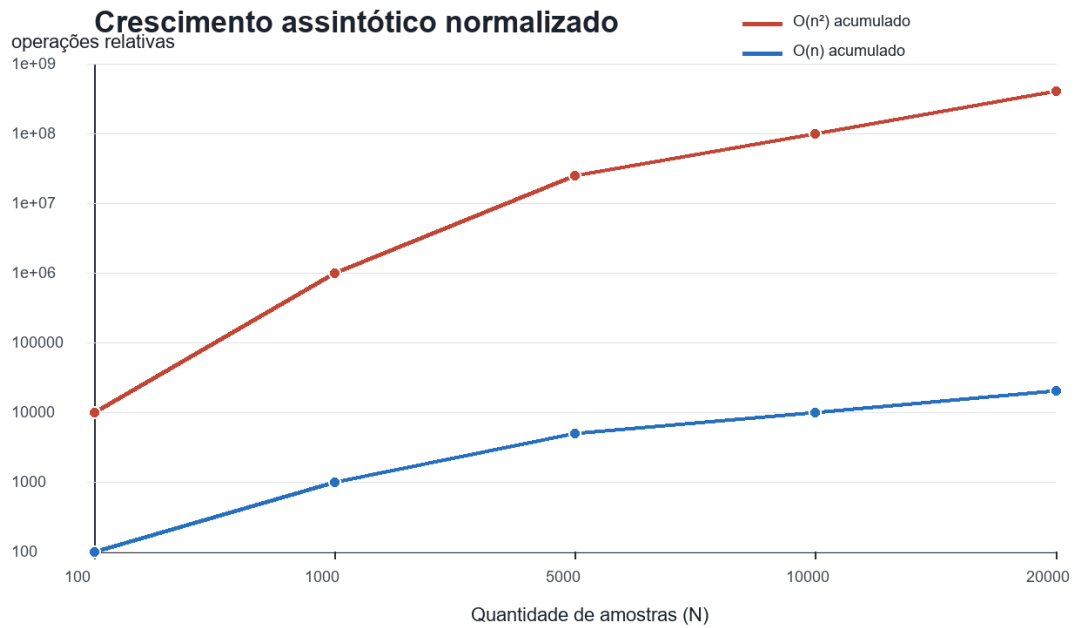


Figura 1: Crescimento assintótico normalizado: o deslocamento acumulado cresce de forma quadrática, enquanto o buffer circular cresce linearmente.

7 Instrumentação

A instrumentação recomendada no firmware mede o tempo gasto na lógica de inserção e a saúde da memória livre:

```

1 unsigned long start = micros();
2 // Logica de insercao de dados
3 unsigned long duration = micros() - start;
4 Serial.printf("Latencia: %lu us | Heap Livre: %u bytes\n",
5               duration,
6               ESP.getFreeHeap());

```

Na vertente ineficiente, o firmware já emite linhas no formato `PERF_DATA`, incluindo número de amostras, latência total, latência média, tempo de publicação, *heap* livre e

falhas de publicação. O notebook `notebooks/perfilamento_buffer_circular.ipynb` gera os gráficos deste relatório e pode ser adaptado para ler diretamente essas linhas seriais quando as medições reais forem coletadas.

8 Resultados

O benchmark local foi executado para $N = 100, 1.000, 5.000, 10.000$ e 20.000 amostras. A Tabela 2 mostra os valores médios gerados pelo notebook.

Tabela 2: Resultados do benchmark controlado.

N	Lista dinâmica ($\mu\text{s}/\text{amostra}$)	Buffer circular ($\mu\text{s}/\text{amostra}$)	Heap circular (KiB)
100	0.10	0.10	199.5
1.000	0.19	0.11	199.5
5.000	0.70	0.12	199.5
10.000	1.39	0.18	199.5
20.000	2.96	0.20	199.5

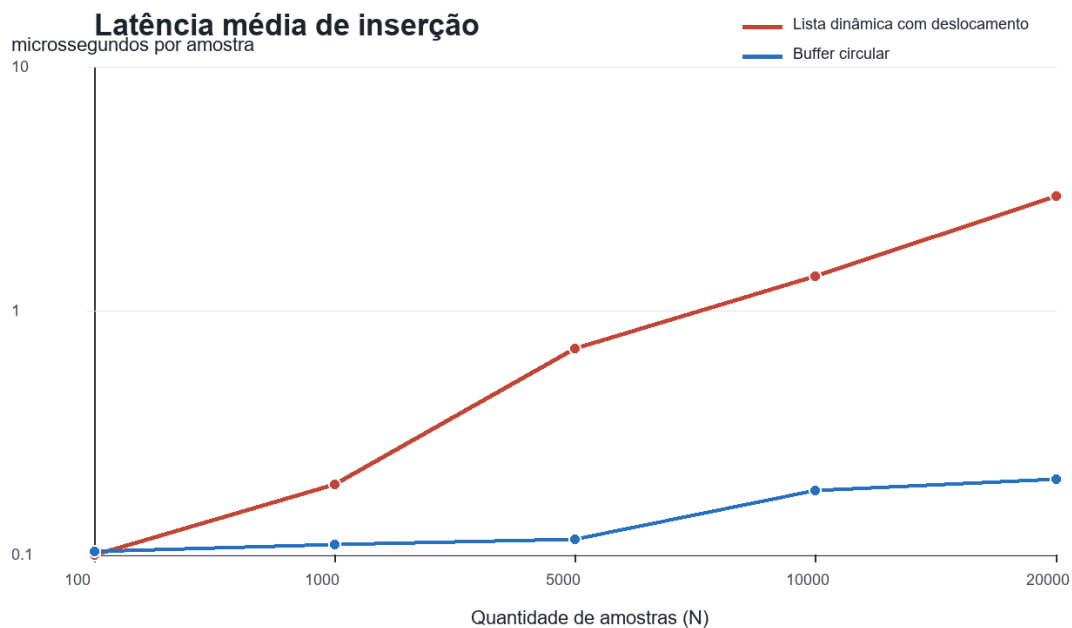


Figura 2: Latência média de inserção. A lista dinâmica aumenta seu custo conforme N cresce; o buffer circular permanece praticamente constante.

A diferença é pequena em $N = 100$, como previsto no requisito. Em $N = 5.000$, o deslocamento já se torna perceptível. Em $N = 20.000$, o custo da lista dinâmica é várias vezes maior do que o do buffer circular. Em um microcontrolador, essa diferença tende a ser mais crítica do que no computador usado para o benchmark, pois há menos CPU, menos memória e maior sensibilidade a interrupções.

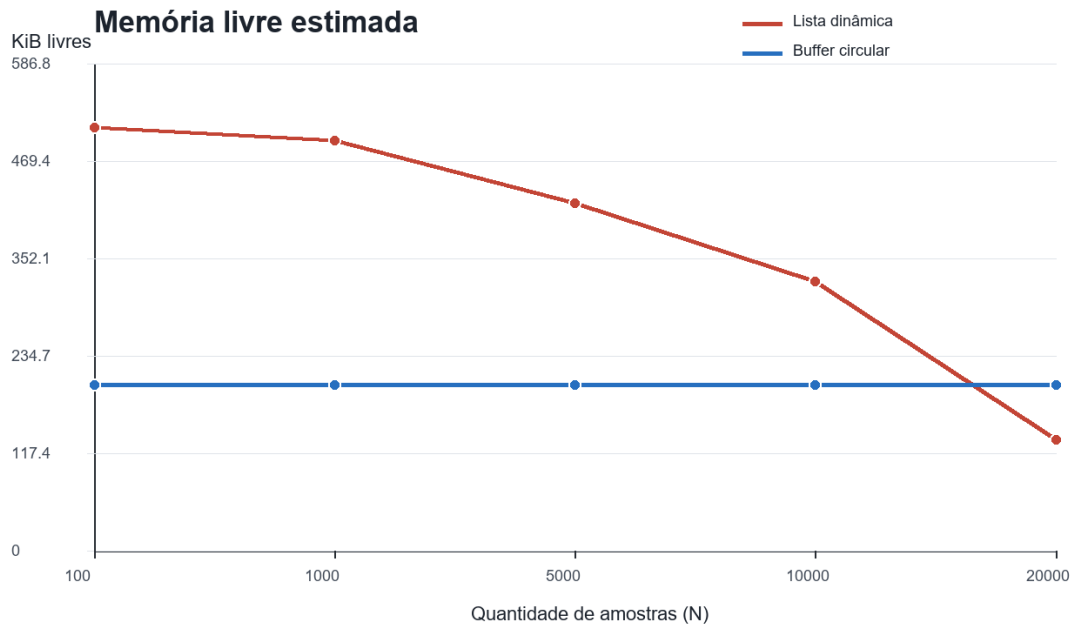


Figura 3: Memória livre estimada. A lista dinâmica reduz progressivamente o *heap* e adiciona risco de fragmentação; o buffer circular usa uma reserva fixa e previsível.

O gráfico de memória mostra uma troca importante: o buffer circular reserva capacidade antecipadamente, mas mantém o consumo estável. A lista dinâmica parece econômica em volumes pequenos, porém seu crescimento progressivo e suas realocações tornam o comportamento menos previsível.

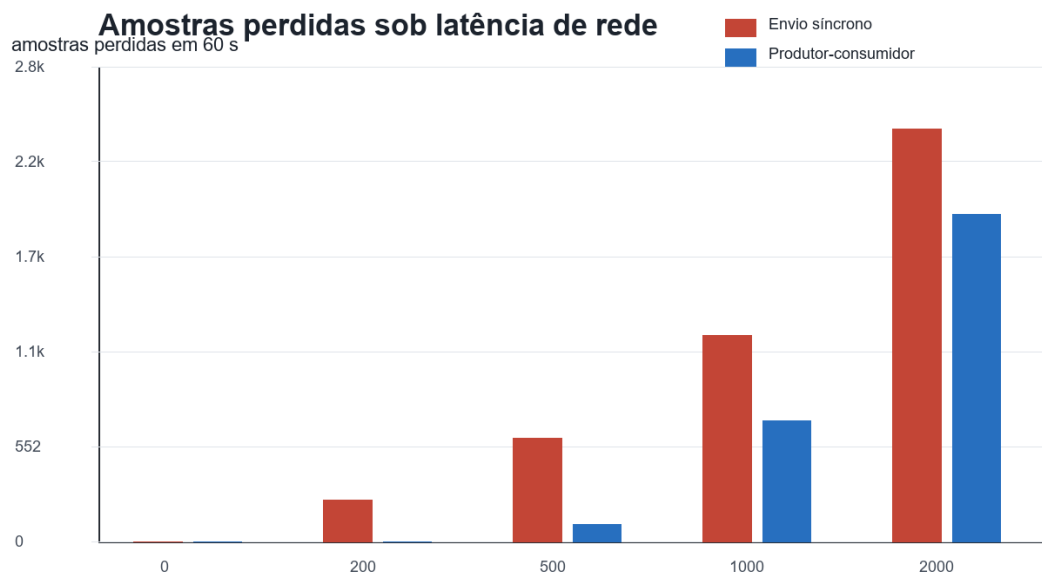


Figura 4: Simulação de perda sob latência de rede. O envio síncrono perde amostras durante bloqueios; o modelo produtor-consumidor absorve parte do atraso no buffer.

Sob instabilidade de rede, o buffer circular funciona como amortecedor. Se a tarefa de MQTT atrasa, a captura continua escrevendo no buffer até o limite de capacidade.

No envio síncrono, a coleta fica presa à latência da rede; por isso as perdas crescem imediatamente quando há bloqueio.

9 Diagnóstico de Memória

O uso de `realloc()` em laço é perigoso em firmware porque o alocador precisa encontrar regiões contíguas cada vez maiores. Mesmo quando há memória livre total suficiente, a memória pode estar fragmentada em blocos pequenos. Nesse cenário, uma nova realocação falha ou demora mais, causando interrupções no fluxo de amostragem.

O buffer circular elimina esse padrão de falha: o bloco é reservado uma vez e reutilizado. A previsibilidade é mais importante do que a economia aparente de memória em cargas pequenas, especialmente quando o sistema precisa operar por longos períodos sem reinicialização.

10 Discussão

Na escala $N = 100$, a diferença entre as duas vertentes é quase imperceptível. Isso explica por que o antipadrão pode parecer aceitável em testes rápidos. Porém, à medida que N cresce, o custo de deslocamento passa a dominar o tempo de execução. Em $N = 5.000$, a latência já aumenta de forma clara; em $N = 20.000$, o sistema fica sujeito a *jitter*, perda de amostras e maior risco de esgotamento do *heap*.

A transmissão MQTT amplia o problema. A rede possui latência variável, e qualquer publicação síncrona dentro do caminho crítico da coleta transforma instabilidade externa em instabilidade do sensor. O modelo produtor-consumidor corrige essa dependência: a tarefa de captura só escreve no buffer, enquanto uma tarefa separada transmite quando a rede está disponível.

Para o projeto de monitoramento WiFi, a mesma conclusão vale para a janela de captura. A coleta passiva precisa ser rápida e previsível; agregação, serialização JSON, MQTT, banco de dados e dashboard devem acontecer fora do trecho crítico.

11 Conclusão

A vertente baseada em buffer circular é superior para telemetria embarcada porque entrega tempo constante por inserção, memória previsível e melhor tolerância a atrasos de rede. A vertente com `realloc()` e `memmove()` tem custo acumulado quadrático, favorece fragmentação do *heap* e transforma latência MQTT em perda de amostras.

O resultado confirma a recomendação de arquitetura: sensores devem produzir eventos em uma estrutura fixa e não bloqueante; consumidores devem processar e transmitir em lote ou em tarefa separada. Essa separação reduz *jitter*, preserva memória e torna o sistema mais escalável.

12 Reprodutibilidade

Os artefatos gerados estão organizados da seguinte forma:

- notebook: `../notebooks/perfilamento_buffer_circular.ipynb`;

- dados do benchmark: `dados/benchmark_resultados.csv`;
- dados da simulação MQTT: `dados/perdas_mqtt.csv`;
- figuras: `figuras/*.png`;
- gerador: `../tools/generate_report_assets.py`.

Para regenerar os gráficos, execute:

```
1 python ../tools/generate_report_assets.py
```

Para compilar o relatório em uma instalação com TeX Live ou Overleaf, use:

```
1 pdflatex relatorio.tex  
2 pdflatex relatorio.tex
```